PLAN TECHNIQUE · CONFIDENTIEL

Present

Plan de développement Backend — de l'application mobile de démonstration à une plateforme de présence scolaire prête pour la production.

Périmètre : Analyse complète des fonctionnalités actuelles (Expo / React Native, stockage local uniquement) et plan d'architecture backend cible.

Préparé pour : Équipe projet Present

Version : 1.0

1. Résumé exécutif

L'application **Present** est aujourd'hui une application mobile Expo / React Native entièrement **côté client** : toutes les données (enseignants, classes, élèves, séances de présence) sont soit codées en dur dans le code source (`data/*.ts`), soit stockées localement sur l'appareil via `AsyncStorage` (bibliothèque `zustand/persist`). Il n'existe **aucun serveur, base de données, API ou service tiers** — le scan de QR code, l'envoi de notifications aux parents, et l'authentification sont tous simulés.

Ce document recense chaque fonctionnalité de l'application, identifie ce qui est aujourd'hui « simulé », et propose une architecture backend cible, un schéma de données, une liste d'endpoints API, une stratégie de notification (WhatsApp), une stratégie de synchronisation hors-ligne, ainsi qu'une feuille de route de mise en œuvre par phases.

> **Constat clé :** le modèle de données local limite actuellement une classe enregistrée par jour au total (toutes classes confondues). Ce comportement est une simplification de démonstration et devra être revu dans le modèle backend pour permettre plusieurs séances (classe + matière) par jour et par enseignant.

2. Table des matières

1. 1. Résumé exécutif
2. 2. Table des matières
3. 3. Inventaire des fonctionnalités & écarts backend
4. 4. Architecture cible proposée
5. 5. Modèle de données (schéma)
6. 6. Endpoints API
7. 7. Stratégie des codes QR
8. 8. Notifications aux parents (WhatsApp)
9. 9. Stockage des fichiers (photos élèves)
10. 10. Authentification & sécurité
11. 11. Stratégie hors-ligne / synchronisation
12. 12. Multi-établissement & scalabilité
13. 13. Plan de migration des données actuelles
14. 14. Feuille de route par phases
15. 15. Questions ouvertes / décisions à prendre

3. Inventaire des fonctionnalités & écarts backend

Chaque écran de l'application a été analysé pour déterminer ce qu'il fait aujourd'hui (côté client uniquement) et ce qu'un backend réel devra fournir.

Fonctionnalité	Comportement actuel (simulé)	Besoin backend
Connexion enseignant `LOGIN.TSX`	L'e-mail saisi est comparé à une liste statique de 5 enseignants ; aucun mot de passe n'est vérifié.	Authentification réelle (e-mail + mot de passe ou lien magique), jetons JWT, gestion de session.
Profil enseignant `PROFILE.TSX`	Statistiques (séances, classes couvertes) calculées à partir du tableau local des séances.	Endpoint de statistiques agrégées côté serveur par enseignant.
Attribution des matières `DATA/TEACHERS.TS`	Chaque enseignant a 1 à N matières codées en dur ; sélection unique appliquée pour la journée.	Table de liaison enseignant↔matière, gérable par un administrateur d'établissement.
Sélection / scan d'une classe `HOME.TSX, CLASS-SCANNER.TSX`	Le « scan » choisit une classe aléatoire dans une liste statique après un délai de 2,5 s.	Vraies classes en base, vrai décodage de QR code, jetons de classe rotatifs.
Carte d'élève / QR élève `GAP` `ID-CARD.TSX`	QR affiché est une grille factice ; l'identifiant élève est généré côté client (`SC-2024-88xx`).	Identifiants élèves durables et uniques, QR signé (anti-duplication de carte), endpoint de résolution QR → élève.
Scan de présence individuel `SCANNER.TSX`	Sélectionne un élève au hasard, simule 15 % d'erreurs, vérifie les doublons localement.	Décodage QR réel, recherche élève, enregistrement idempotent avec horodatage serveur.
Registre de présence `REGISTER.TSX`	Onglets Présents / Pas encore arrivés, marquage manuel, champ de recherche non fonctionnel.	CRUD de présence, endpoint de recherche d'élèves côté serveur.
Marquage « Absent » explicite `STORE.MARKSTUDENTABSENT`	Distingue « non comptabilisé » de « marqué absent » via un simple booléen local.	Statut de présence en base : `pending | present | absent` .
Inscription d'un élève `GAP CRITIQUE` `REGISTER-STUDENT.TSX`	Le bouton « Enregistrer » exécute `router.back()` — **aucune donnée n'est réellement sauvegardée**, y compris la photo.	Endpoint de création d'élève, upload de photo, stockage du contact WhatsApp du tuteur.
Résultats de scan `SCAN-RESULT/*.TSX`	L'écran de succès affiche « Parent averti » sans rien envoyer.	Déclenchement réel d'une notification (WhatsApp/SMS) avec statut de livraison.

Historique HISTORY.TSX	Liste les séances stockées localement ; une seule séance possible par jour au total.	Historique complet par établissement/enseignant/classe, pagination, filtre par date.
Persistance des données GAP CRITIQUE	`AsyncStorage` uniquement — données perdues à la désinstallation, non partagées entre appareils.	Base de données serveur, compte lié à l'enseignant, synchronisation multi-appareil.

4. Architecture cible proposée

Deux options sont envisageables selon le budget, le délai et la taille de l'équipe :

Option A — Backend-as-a-Service (recommandé pour le MVP)

Supabase (Postgres managé + Auth + Storage + Realtime + Row Level Security) permet de livrer un backend fonctionnel en quelques semaines sans gérer d'infrastructure serveur.

- **Auth** : Supabase Auth (e-mail/mot de passe, gestion de session, JWT).
- **Base de données** : Postgres managé avec Row Level Security (isolation par établissement).
- **Stockage** : Supabase Storage pour les photos d'élèves.
- **Temps réel** : abonnement en direct aux mises à jour de présence (utile pour un tableau de bord administrateur).
- **Fonctions** : Edge Functions pour la logique métier (validation QR, déclenchement des notifications WhatsApp).

Option B — Backend sur mesure

Recommandé si l'application doit évoluer vers un produit SaaS multi-établissements à grande échelle.

- **API** : Node.js + NestJS (ou Fastify), architecture modulaire par domaine (auth, écoles, présence).
- **Base de données** : PostgreSQL (via Prisma ou TypeORM).
- **File d'attente** : Redis + BullMQ pour les tâches asynchrones (notifications).
- **Stockage objet** : S3 (ou compatible) pour les photos.
- **Hébergement** : conteneurs (Docker) sur un fournisseur cloud, avec attention portée à la résidence des données pour la RDC/Afrique si requis.

> **Recommandation :** démarrer avec l'Option A (Supabase) pour valider le produit rapidement, puis migrer vers l'Option B seulement si les besoins de personnalisation ou d'échelle le justifient. Le schéma de données proposé (section 5) est conçu pour fonctionner avec les deux options.

5. Modèle de données (schéma)

Le schéma introduit un `school_id` sur chaque table dès le départ pour permettre un modèle multi-établissements, même si le lancement initial ne couvre qu'un seul établissement.

schools

Colonne	Type	Description
id	uuid, PK	Identifiant de l'établissement
name	text	Nom de l'établissement
country	text	Ex. « CD » (RD Congo)
timezone	text	Ex. « Africa/Kinshasa »
created_at	timestamptz	

teachers

Colonne	Type	Description
id	uuid, PK	
school_id	uuid, FK	
full_name	text	
email	text, unique	Utilisé pour la connexion
password_hash	text	Géré par le fournisseur d'authentification
avatar_url	text	
role	enum	`teacher | admin`

subjects & teacher_subjects

Table	Colonnes	Description
subjects	id, school_id, code, label	Ex. `MATHS / Mathématiques`
teacher_subjects	teacher_id, subject_id	Relation N↔N

classes

Colonne	Type	Description
id	uuid, PK	
school_id	uuid, FK	
name	text	Ex. « Grade 5B »

qr_token	text, unique	Jeton rotatif encodé dans le QR de la classe

students

Colonne	Type	Description
id	uuid, PK	
class_id	uuid, FK	
full_name	text	
student_code	text, unique	Identifiant lisible imprimé sur la carte
qr_token	text, unique	Jeton signé encodé dans le QR de l'élève
photo_url	text	
guardian_name	text	
guardian_whatsapp	text	Avec indicatif pays

sessions

Colonne	Type	Description
id	uuid, PK	
class_id	uuid, FK	
teacher_id	uuid, FK	
subject_id	uuid, FK	
date	date	
started_at	timestamptz	

Contrainte d'unicité recommandée : (class_id, subject_id, date) — remplace la limite actuelle d'« une seule séance par jour » par une contrainte correcte par classe et par matière.

attendance_records

Colonne	Type	Description
id	uuid, PK	
session_id	uuid, FK	
student_id	uuid, FK	
status	enum	`pending | present | absent`
marked_at	timestamptz	
method	enum	`qr_scan | manual`

notifications

Colonne	Type	Description
id	uuid, PK	
attendance_record_id	uuid, FK	
channel	enum	`whatsapp | sms`
status	enum	`queued | sent | delivered | failed`
sent_at	timestamptz	

6. Endpoints API

Aperçu des endpoints REST nécessaires, regroupés par domaine fonctionnel.

Authentification

Méthode	Route	Description
POST	`/auth/login`	Connexion enseignant (e-mail + mot de passe)
POST	`/auth/refresh`	Rafraîchir le jeton d'accès
POST	`/auth/logout`	Invalider la session

Enseignants & matières

Méthode	Route	Description
GET	`/teachers/me`	Profil de l'enseignant connecté
GET	`/teachers/me/subjects`	Matières assignées
GET	`/teachers/me/stats`	Séances/classes couvertes

Classes & élèves

Méthode	Route	Description
GET	`/classes`	Liste des classes de l'établissement
GET	`/classes/resolve-qr`	Résoudre un jeton QR de classe scanné
GET	`/classes/:id/roster`	Liste des élèves d'une classe
POST	`/students`	Inscrire un nouvel élève (multipart, photo incluse)
GET	`/students/:id`	Détail d'un élève (pour la carte ID)
GET	`/students/resolve-qr`	Résoudre un jeton QR élève scanné

Séances & présence

Méthode	Route	Description
POST	`/sessions`	Ouvrir une séance (classe + matière + date)
GET	`/sessions/today`	Séances du jour pour l'enseignant connecté
POST	`/sessions/:id/attendance/scan`	Enregistrer une présence via QR élève
POST	`/sessions/:id/attendance/:studentId/present`	Marquer présent manuellement
POST	`/sessions/:id/attendance/:studentId/absent`	Marquer absent manuellement
GET	`/sessions/:id/attendance?search=`	Liste + recherche des présences

Historique

Méthode	Route	Description
GET	`/history?teacherId=&from=&to=`	Historique paginé des séances
GET	`/history/:sessionId`	Détail d'une séance passée

7. Stratégie des codes QR

QR de classe

Encode `{ schoolId, classId, rotatingToken }`. Le jeton rotatif est régénéré périodiquement (ex. chaque trimestre) pour permettre de réimprimer les affiches de classe sans compromettre la sécurité si une image est partagée.

QR élève (carte d'identité)

Encode `{ studentId, signature }` où `signature` est un HMAC signé avec une clé secrète serveur. Cela empêche la fabrication d'une fausse carte ou la copie d'un identifiant simple. En cas de perte de carte, un nouveau jeton peut être émis, invalidant l'ancien.

> La grille QR actuellement affichée dans `components/IDCard.tsx` est purement décorative (données statiques) — elle devra être remplacée par un rendu de QR réel (ex. bibliothèque `react-native-qrcode-svg`) encodant le jeton signé retourné par l'API.

8. Notifications aux parents (WhatsApp)

Le numéro WhatsApp du tuteur est déjà collecté dans l'écran d'inscription (`register-student.tsx`), avec indicatif pays — l'intégration cible naturelle est la **WhatsApp Business Cloud API** (Meta).

1. À chaque enregistrement de présence (`status = present`), une tâche est ajoutée à une file d'attente asynchrone.
2. Un worker envoie un message via un modèle pré-approuvé par Meta (ex. « {élève} est arrivé(e) à l'école à {heure} »).
3. Le statut de livraison (`queued / sent / delivered / failed`) est stocké et peut être affiché dans l'application.

> **Délai à anticiper :** la vérification d'entreprise Meta et l'approbation des modèles de message WhatsApp peuvent prendre plusieurs semaines — à démarrer tôt dans le calendrier du projet. Une solution de repli par SMS (ex. Twilio, Africa's Talking) est recommandée le temps de l'approbation.

9. Stockage des fichiers (photos élèves)

La sélection de photo via `expo-image-picker` fonctionne déjà côté client, mais l'image reste uniquement sur l'appareil (URI locale, jamais téléversée). Il faut :

- Un endpoint d'upload (multipart ou URL signée pré-autorisée) retournant une `photo_url` durable.
- Redimensionnement/compression côté client avant envoi (déjà partiellement géré par `allowsEditing` et `quality: 0.8`).
- Politique de rétention et de suppression conforme à la protection des données des mineurs.

10. Authentification & sécurité

- Remplacer la correspondance d'e-mail factice par une authentification réelle (mot de passe hashé, ou lien magique).
- Jetons d'accès JWT de courte durée + jeton de rafraîchissement.
- Contrôle d'accès par rôle (enseignant vs administrateur d'établissement).
- Limitation de débit (rate limiting) sur les endpoints de scan pour éviter les abus.
- Protection contre le rejeu des jetons QR (fenêtre de validité, invalidation après réémission).
- Minimisation des données collectées sur les mineurs ; consentement du tuteur capturé à l'inscription ; droit à l'effacement.

11. Stratégie hors-ligne / synchronisation

La connectivité réseau dans certains établissements en RDC peut être intermittente. L'application doit conserver un modèle « hors-ligne d'abord » :

- Le magasin local (`zustand + AsyncStorage`) devient une **file d'attente d'écriture** plutôt que la source de vérité.
- Chaque action de présence est créée avec un identifiant UUID côté client pour permettre un *upsert idempotent* côté serveur (évite les doublons en cas de nouvel essai réseau).
- Synchronisation en arrière-plan dès que la connectivité revient ; indicateur visuel de synchronisation en attente dans l'interface.

12. Multi-établissement & scalabilité

Même si le lancement initial ne concerne qu'un seul établissement, le schéma proposé inclut `school_id` sur chaque table dès le départ. Cela évite une migration coûteuse si l'application est déployée dans plusieurs écoles (modèle SaaS), et permet d'isoler les données par établissement via des règles de sécurité au niveau des lignes (Row Level Security).

13. Plan de migration des données actuelles

Source actuelle (client)	Destination backend
`data/classes.ts`	Script de seed → table `classes` + `students`
`data/teachers.ts`	Script de seed → table `teachers` + `teacher_subjects`
`data/subjects.ts`	Script de seed → table `subjects`
`store/useAppStore.ts`	Conserver comme cache client, mais chaque action (`checkInClass` , `markStudentPresent` , `markStudentAbsent` , `setTeacher` , `setActiveSubject`) appelle désormais l'API, avec mise à jour optimiste de l'interface.

14. Feuille de route par phases

Phase 0 — Fondations 1–2 semaines

Choix définitif de la stack (Option A ou B), provisionnement de l'infrastructure, schéma de base de données et migrations, mise en place de l'authentification.

Phase 1 — CRUD de base 2–3 semaines

Endpoints établissements/enseignants/classes/élèves/matières. Connexion réelle des écrans `login.tsx` et `register-student.tsx`.

Phase 2 — Moteur de présence 2 semaines

Endpoints séances + présence, émission/validation des jetons QR, connexion des écrans `class-scanner.tsx`, `scanner.tsx`, `register.tsx`.

Phase 3 — Notifications 1–2 semaines

Intégration WhatsApp Business Cloud API + worker asynchrone, statut de livraison affiché dans l'application.

Phase 4 — Historique, statistiques & synchronisation hors-ligne 2 semaines

Endpoints d'historique, statistiques de profil, file d'attente hors-ligne côté client et résolution des conflits.

Phase 5 — Consolidation 1–2 semaines

Revue de sécurité, limitation de débit, sauvegardes, observabilité/journalisation, tests de charge.

Estimation totale : ~10 à 14 semaines pour un MVP, avec 1 à 2 ingénieurs backend. La Phase 3 (approbation Meta) doit démarrer le plus tôt possible en parallèle des autres phases.

15. Questions ouvertes / décisions à prendre

- L'application doit-elle supporter plusieurs établissements dès le lancement, ou un seul établissement pour commencer ?
- Y a-t-il un budget/accès pour la vérification d'entreprise Meta (WhatsApp Business API) ? Sinon, quel canal de repli (SMS) privilégier ?
- Préférence entre une solution managée (Supabase) pour aller vite, ou un backend sur mesure pour plus de contrôle à long terme ?
- Existe-t-il une contrainte de résidence des données (hébergement en Afrique/RDC) pour des raisons réglementaires ou de latence ?
- Qui gérera l'inscription des élèves et l'impression des cartes ID en conditions réelles (l'administration de l'école, ou chaque enseignant) ?